DAA CP

Fair and Capacity-Constrained Bipartite Matching

TY-F9

October 2026

Abstract

This document defines a domain-independent allocation model for fair, capacity-constrained bipartite matching and specifies how a student–project application constructs its eligibility graph and edge utilities from weighted skills. The model separates eligibility from optimization: application-specific rules determine which assignments are allowed, while the matching algorithms operate only on the resulting graph, utilities, and resource capacities.

1 Introduction

The project studies the assignment of agents to resources under resource capacity constraints. The motivating application is the allocation of students to projects. The core formulation is domain-independent: agents and resources can represent other entities without changing the matching algorithms.

The model supports three allocation objectives: maximum cardinality, maximum total utility, and a selected lexicographic fairness objective. In the student–project application, skill information determines whether a student is eligible for a project and, if eligible, how suitable that assignment is.

2 Domain-Independent Allocation Model

Let

$$A = \{0, \dots, n-1\} \qquad \text{and} \qquad R = \{0, \dots, m-1\}$$

be the sets of agents and resources, respectively. Let

$$E \subseteq A \times R$$

be the set of eligible agent–resource pairs. Each resource $j \in R$ has a non-negative integer capacity c_j. Every eligible edge $(i, j) \in E$ has an integer utility $u_{ij} \in \{0, 1, \dots, 100\}$.

For each eligible pair $(i, j) \in E$, define the binary decision variable

$$x_{ij} = \begin{cases} 1, & \text{if agent } i \text{ is assigned to resource } j, \\ 0, & \text{otherwise.} \end{cases}$$

An allocation is feasible if no agent is assigned to more than one resource and no resource exceeds its capacity:

$$\sum_{j:(i,j)\in E} x_{ij} \le 1 \qquad \forall i \in A, \tag{1}$$

$$\sum_{i:(i,j)\in E} x_{ij} \le c_j \qquad \forall j \in R, \tag{2}$$

$$x_{ij} \in \{0, 1\} \qquad \forall (i, j) \in E. \tag{3}$$

An absent edge represents an ineligible assignment. An existing edge may have utility zero; therefore, an edge with zero utility must not be treated as a missing edge. The model does not require every agent to be assigned.

3 Allocation Objectives

3.1 Maximum Cardinality

The maximum-cardinality objective maximizes the total number of assignments:

$$\max \quad \sum_{(i,j)\in E} x_{ij}. \tag{4}$$

3.2 Maximum Total Utility

The maximum-weight objective maximizes the sum of the utilities of the selected assignments:

$$\max \quad \sum_{(i,j)\in E} u_{ij}x_{ij}. \tag{5}$$

3.3 Selected Lexicographic Fairness Objective

The selected fairness objective is lexicographic. It prioritizes the objectives in the following order:

1. maximize the number of assignments;
2. among allocations with maximum cardinality, maximize the minimum utility of the assigned agents;
3. among allocations tied on the first two criteria, maximize total utility.

For a non-empty allocation X, let $u_{\min}(X)$ denote the minimum utility among its assigned edges. The reported fairness score is

$$F(X) = \begin{cases} \dfrac{u_{\min}(X)}{100}, & \text{if } X \text{ is non-empty,} \\ 0, & \text{if } X \text{ is empty.} \end{cases} \tag{6}$$

This is an individual max-min utility objective. It is not a group-parity, envy-freeness, or stable-matching objective. Student group labels may be carried as instance metadata, but they do not affect the selected fairness objective.

4 Student–Project Eligibility and Utility

The application converter constructs the eligibility graph and utilities before an instance is passed to a matching algorithm. A project p defines a non-empty set of uniquely named skills K_p. Each skill $k \in K_p$ has a positive, finite relative importance weight q_{pk} and a Boolean mandatory flag. Student skills are represented as sets of skill names.

4.1 Normalizing Project Skill Weights

For each project p, normalize the relative weights as

$$w_{pk} = \frac{q_{pk}}{\sum_{\ell\in K_p} q_{p\ell}}, \qquad k \in K_p. \tag{7}$$

Thus, $w_{pk} > 0$ and

$$\sum_{k \in K_p} w_{pk} = 1.$$

The weights express the relative importance of project skills and need not be provided in normalized form in the input data.

4.2 Weighted Skill Coverage

Let S_s denote the set of skills possessed by student s. Define the indicator function

$$\mathbf{1}[k \in S_s] = \begin{cases} 1, & \text{if student } s \text{ possesses skill } k, \\ 0, & \text{otherwise.} \end{cases}$$

The weighted coverage of student s for project p is

$$\text{coverage}(s, p) = 100 \sum_{k \in K_p} w_{pk} \mathbf{1}[k \in S_s]. \tag{8}$$

Coverage lies in the interval $[0, 100]$. Mandatory skills contribute to this coverage score as well as imposing a separate eligibility condition.

4.3 Eligibility and Edge Utility

Let $M_p \subseteq K_p$ be the set of mandatory skills for project p, and let t_p be the minimum coverage threshold for that project. The default threshold is 35, and valid threshold values lie in $[0, 100]$. A student–project pair is eligible if and only if the student possesses every mandatory skill and meets the coverage threshold:

$$(s, p) \in E \quad \iff \quad M_p \subseteq S_s \ \wedge \ \text{coverage}(s, p) \geq t_p. \tag{9}$$

For an eligible pair, the integer edge utility is the coverage rounded to the nearest integer, with half values rounded upward:

$$u_{sp} = \text{round_half_up}\big(\text{coverage}(s, p)\big). \tag{10}$$

Consequently, edge utilities lie in $[0, 100]$. Eligibility is determined before matching optimization, and an eligible edge can have utility zero when the configured threshold permits it.

4.4 Input Validation

The converter rejects invalid project skill configurations, including:

- a missing or empty project skill list;
- duplicate skill names within a project;
- weights that are non-positive or non-finite;
- thresholds outside $[0, 100]$; and
- more than three mandatory skills for a project.

Project capacities and student group labels are translated into common instance metadata. Group labels do not alter eligibility, utility, or the selected fairness objective.

5 Separation Between Application and Algorithms

The student–project converter is responsible for interpreting skill names, project-specific weights, mandatory flags, coverage thresholds, capacities, and other application data. It constructs the eligible edge set E and the utility u_{sp} for each eligible pair. After construction, the algorithms consume only the graph, edge utilities, and resource capacities. They do not contain student-specific or project-specific skill logic.

This separation allows the same matching algorithms to be used in other domains by supplying a different eligibility graph, utility values, and capacities.